

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Курсовая работа
по курсу «Параллельная обработка данных»

Обратная трассировка лучей (Ray Tracing) на GPU.

Выполнил: Д.Н. Путилин
Группа: М8О-407Б-21
Преподаватель: А.Ю. Морозов

Москва, 2024

Условие

Цель работы: использование GPU для создания фотореалистической визуализации. Рендеринг полужеркальных и полупрозрачных правильных геометрических тел. Получение эффекта бесконечности. Создание анимации.

Вариант 2. Тетраэдр, гексаэдр, додекаэдр.

Сцена. Прямоугольная поверхность (пол), над которой расположены три платоновых тела. Сверху находятся источники света.

Камера. Камера выполняет облет сцены согласно следующим законам. В цилиндрических координатах положение и направление камеры определяется как:

$$r_c(t) = r_c^0 + A_c^r \sin(\omega_c^r \cdot t + p_c^r)$$

$$z_c(t) = z_c^0 + A_c^z \sin(\omega_c^z \cdot t + p_c^z)$$

$$\varphi_c(t) = \varphi_c^0 + \omega_c^\varphi t$$

$$r_n(t) = r_n^0 + A_n^r \sin(\omega_n^r \cdot t + p_n^r)$$

$$z_n(t) = z_n^0 + A_n^z \sin(\omega_n^z \cdot t + p_n^z)$$

$$\varphi_n(t) = \varphi_n^0 + \omega_n^\varphi t$$

Требуется реализовать алгоритм обратной трассировки лучей с использованием CUDA. Выполнить покадровый рендеринг сцены. Для устранения эффекта «зубчатости» выполнить сглаживание (SSAA). Полученный набор кадров склеить в анимацию любым доступным программным обеспечением. Подобрать параметры сцены, камеры и освещения таким образом, чтобы получить наиболее красочный результат. Провести сравнение производительности gpu и cpu (т.е. дополнительно нужно реализовать алгоритм без использования cuda).

Входные данные. Программа принимает на вход следующие параметры:

1. Количество кадров
2. Путь к выходным изображениям (строка со спецификатором %d)
3. Разрешение экрана и угол обзора в градусах по горизонтали
4. Параметры движения камеры (коэффициенты из формул выше)
5. Параметры трех тел: центр тела, цвет тела (нормированный), радиус (подразумевается радиус описанной сферы)
6. Параметры пола: четыре точки, оттенок цвета

7. Параметры источника света: положение и цвет (нормированный)
8. Квадратный корень из количества лучей на один пиксель для SSAA.

Выходные данные. В соответствующие файлы нужно записать полученные картинки в бинарном виде (как было в лабораторных работах).

Программное и аппаратное обеспечение

Compute capability : 7.5

Name : Tesla T4

Total Global Memory : 15835660288

Shared memory per block : 49152

Registers per block: 65536

Warp size : 32

Max threads per block : (1024, 1024, 64)

Max block : (2147483647, 65535, 65535)

Total constant memory : 65536

Multiprocessors count : 40

Метод решения

Для реализации трассировки лучей в этой программе используется алгоритм трассировки пути с поддержкой прозрачности, отражений и текстурирования.

Основные этапы решения:

1. Поиск пересечения луча с полигонами:

Используется метод Мёллера-Трумборе для нахождения пересечения луча с треугольниками, которые составляют объекты сцены. Алгоритм возвращает точку пересечения и расстояние до неё.

2. Освещение и затенение:

Вычисляется освещение с учётом нескольких источников света. Проверяется, находится ли точка пересечения в тени, с учётом прозрачных объектов. Освещение рассчитывается по модели Ламберта для диффузного освещения.

3. Текстурирование:

Если полигон имеет текстуру, выполняется билинейная интерполяция для выборки цвета текстуры с использованием барицентрических координат u и v .

4. Обработка прозрачности:

Для прозрачных объектов луч продолжается через поверхность, а интенсивность света уменьшается пропорционально коэффициенту прозрачности.

5. Обработка отражений:

Для отражающих объектов рассчитывается направление отражённого луча, который продолжается от точки пересечения, с уменьшением интенсивности света в соответствии с коэффициентом отражения.

6. Рекурсивная трассировка:

Алгоритм поддерживает заданную глубину рекурсии для обработки многократных отражений и прозрачных объектов.

7. SSAA:

Применяется суперсемплинг (SSAA), при котором для каждого пикселя экрана генерируется несколько лучей с последующим усреднением цвета для сглаживания границ объектов.

Описание программы

Программа реализует **трассировку лучей** для создания фотореалистичных изображений с поддержкой прозрачности, отражений и текстурирования. Основные функции программы:

1. Чтение входных параметров:

Программа считывает конфигурацию сцены, включая размеры экрана, параметры камеры, положения объектов (тетраэдра, гексаэдра, додекаэдра и пола), параметры освещения и параметры рендеринга (прозрачность, отражение, суперсемплинг и глубину рекурсии).

1. Генерация объектов сцены:

Создаются полигоны, представляющие объекты сцены:

- Тетраэдр, гексаэдр и додекаэдр создаются с помощью соответствующих функций.
- Пол может быть одноцветным или с текстурой, если указана текстура.

3. Рендеринг сцены:

Используется алгоритм трассировки пути с учётом:

- **Прямого освещения** с несколькими источниками света.
- **Теней**, включая частичные тени для прозрачных объектов.
- **Отражений** для зеркальных поверхностей.
- **Текстурирования** с выборкой цвета из текстуры.

4. Суперсемплинг (SSAA):

Применяется суперсемплинг для сглаживания изображения.

5. Поддержка GPU:

Основные вычисления могут выполняться как на CPU, так и на GPU, что позволяет значительно ускорить рендеринг при большом количестве кадров и сложной сцене.

6. Вывод изображения:

Сгенерированные изображения сохраняются в указанной папке в виде бинарных файлов. Для каждого кадра выводится время рендеринга.

7. Режимы работы:

- `--default`: Выводит наиболее красочную конфигурацию входных данных.
- `--cpu`: Выполняет рендеринг на CPU.
- `--gpu`: Выполняет рендеринг на GPU.
- `--texture <file>`: Загружает текстуру для пола из указанного файла.

Исследовательская часть и результаты

Для оценки производительности программы сравним время рендеринга одного кадра при использовании графического процессора (`kernel<<<16, 16>>>`) и

центрального процессора. При этом будем рендерить одну и ту же сцену, изменяя разрешение изображения, что пропорционально увеличивает количество обрабатываемых лучей. В ходе эксперимента замерим только чистое время рендеринга кадра, исключая задержки, связанные с передачей данных между видеокартой и оперативной памятью.

Глубина рекурсии: 2

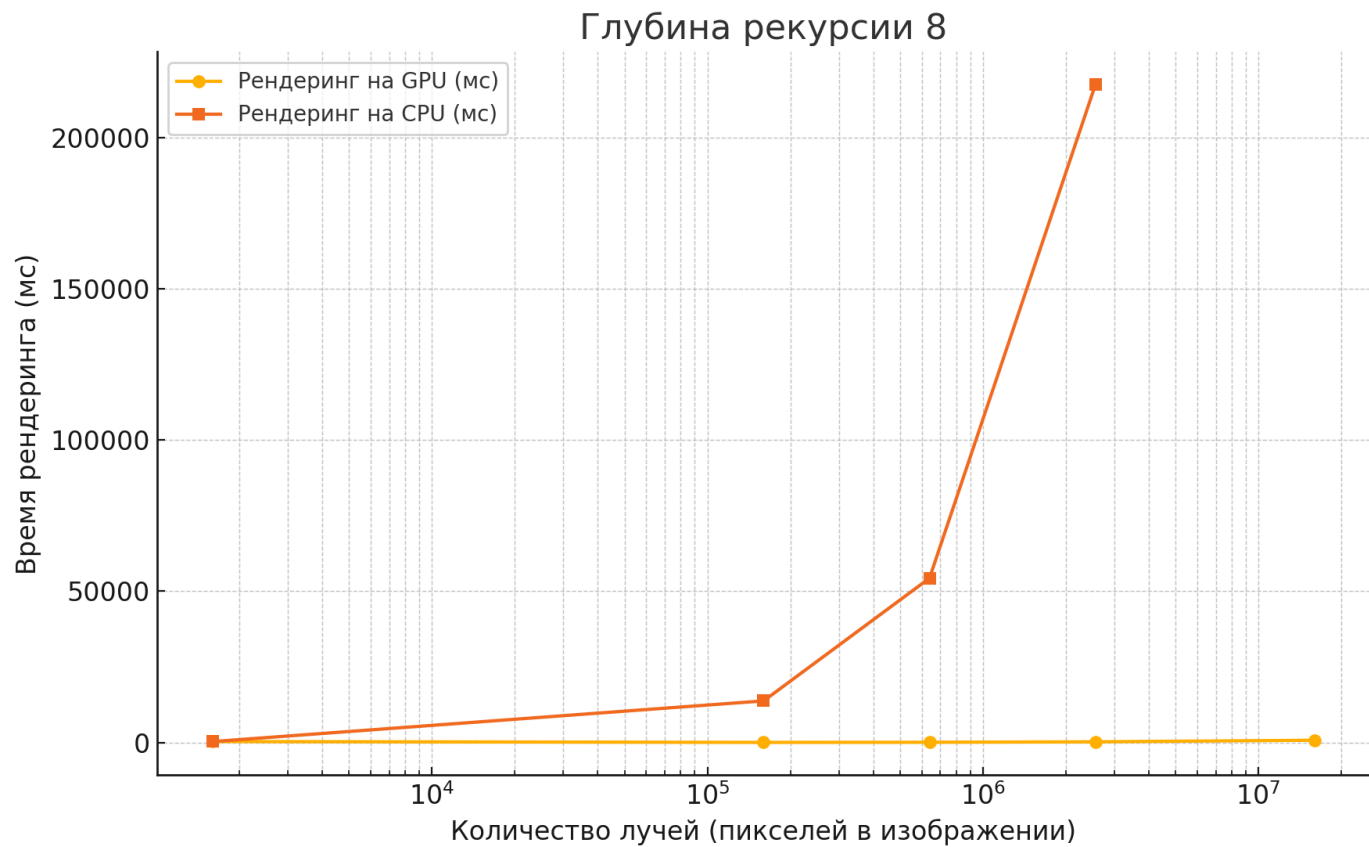
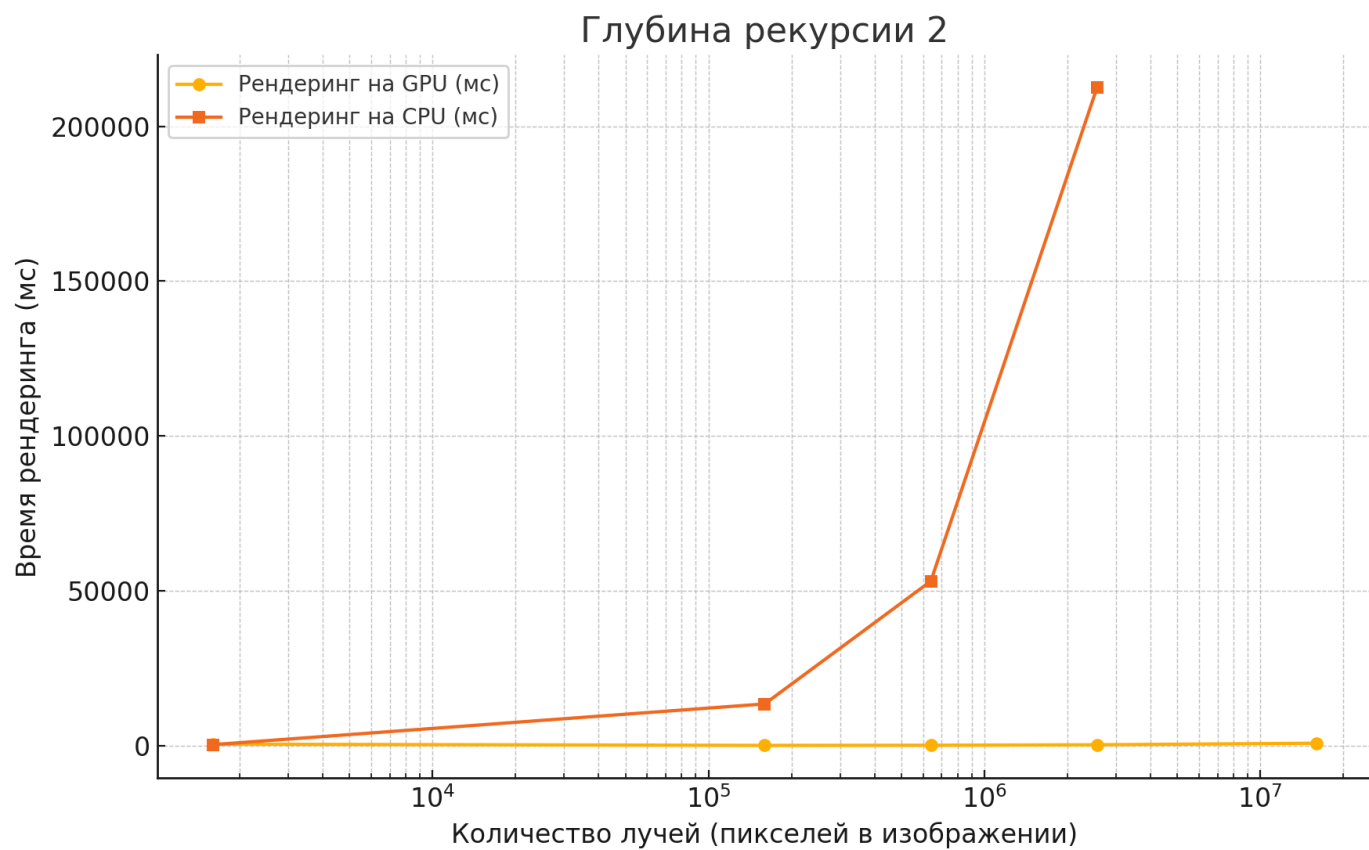
Количество лучей (пикселей в картинке)	Рендеринг 1 кадра на GPU, мс	Рендеринг 1 кадра на CPU, мс
1600 (10x10)	429.779	259.231
160000 (100x100)	18.4791	13415
640000 (200x200)	60.9371	53086.6
2560000 (400x400)	218.69	212471
16000000 (1000x1000)	724.477	-

Глубина рекурсии: 8

Количество лучей (пикселей в картинке)	Рендеринг 1 кадра на GPU, мс	Рендеринг 1 кадра на CPU, мс
1600 (10x10)	307.631	292.795
160000 (100x100)	21.264	13740
640000 (200x200)	62.0014	54313
2560000 (400x400)	214.907	217422
16000000 (1000x1000)	729.845	-

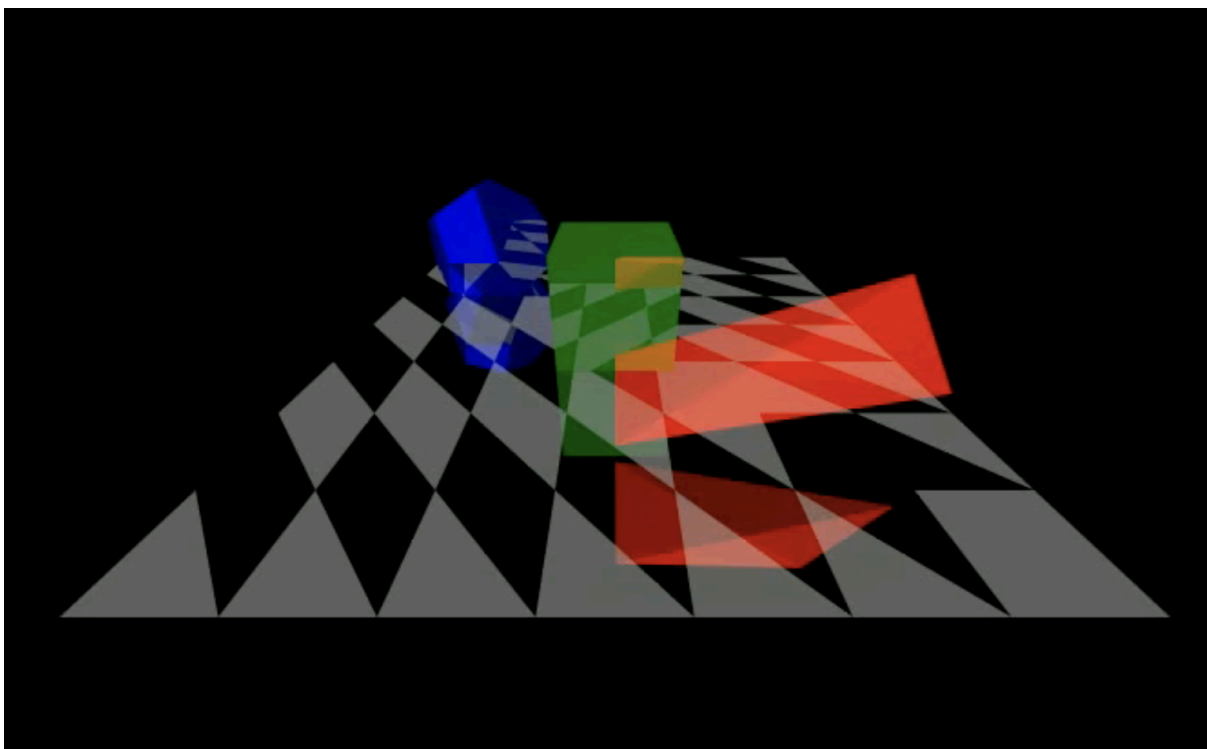
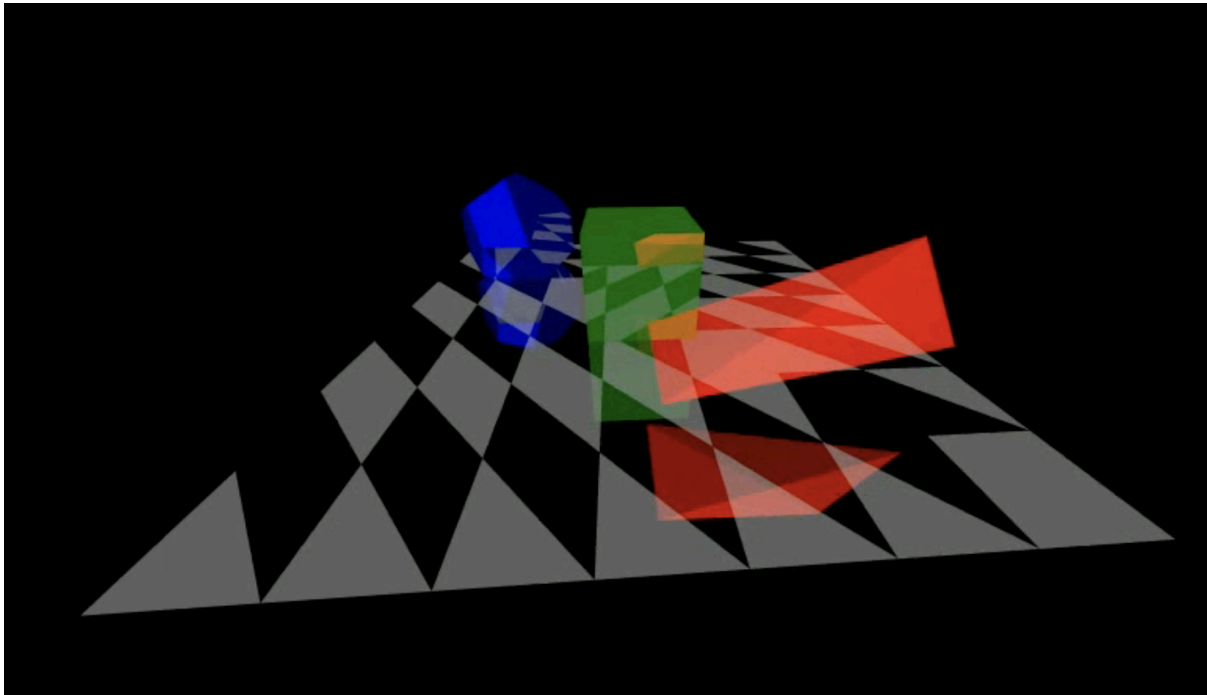
Использование видеокарт значительно уменьшает время работы!

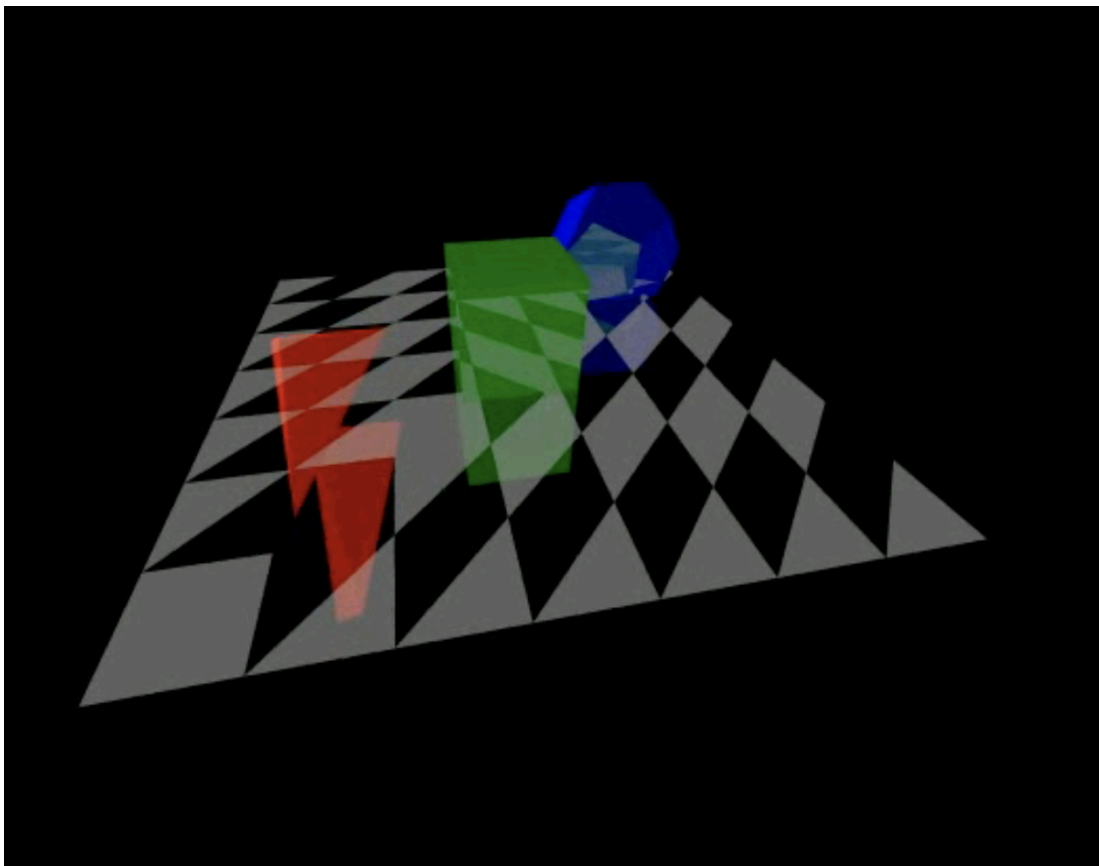
Графики представленных экспериментов:



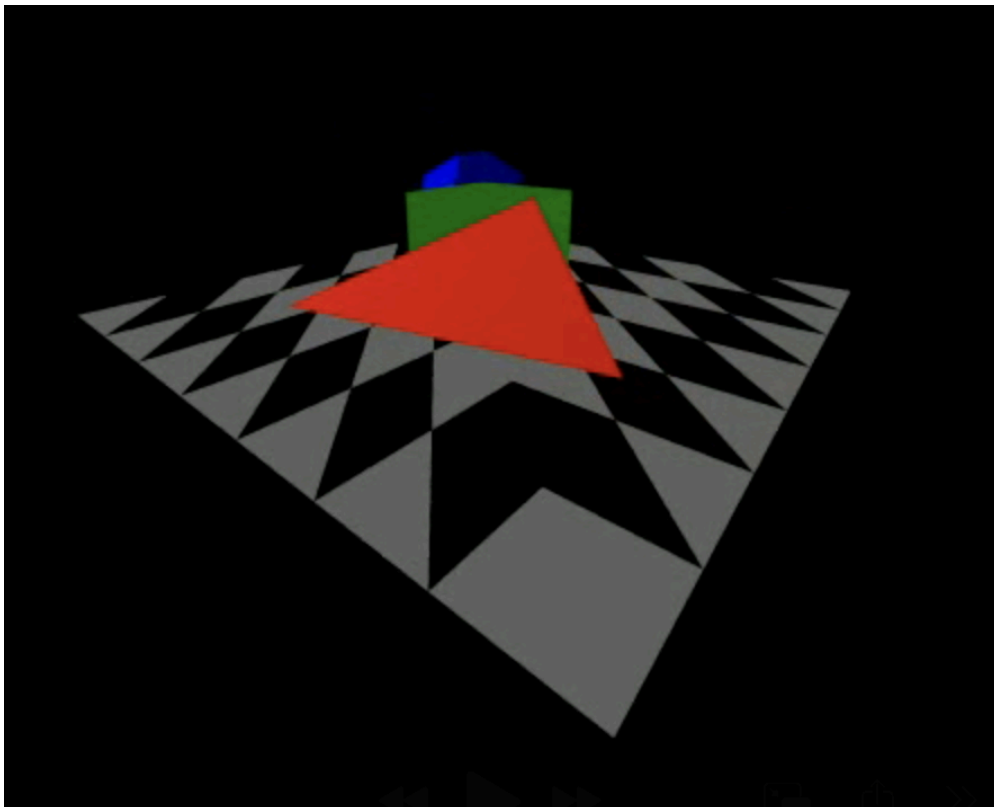
Продемонстрирую скриншоты полученной анимации:

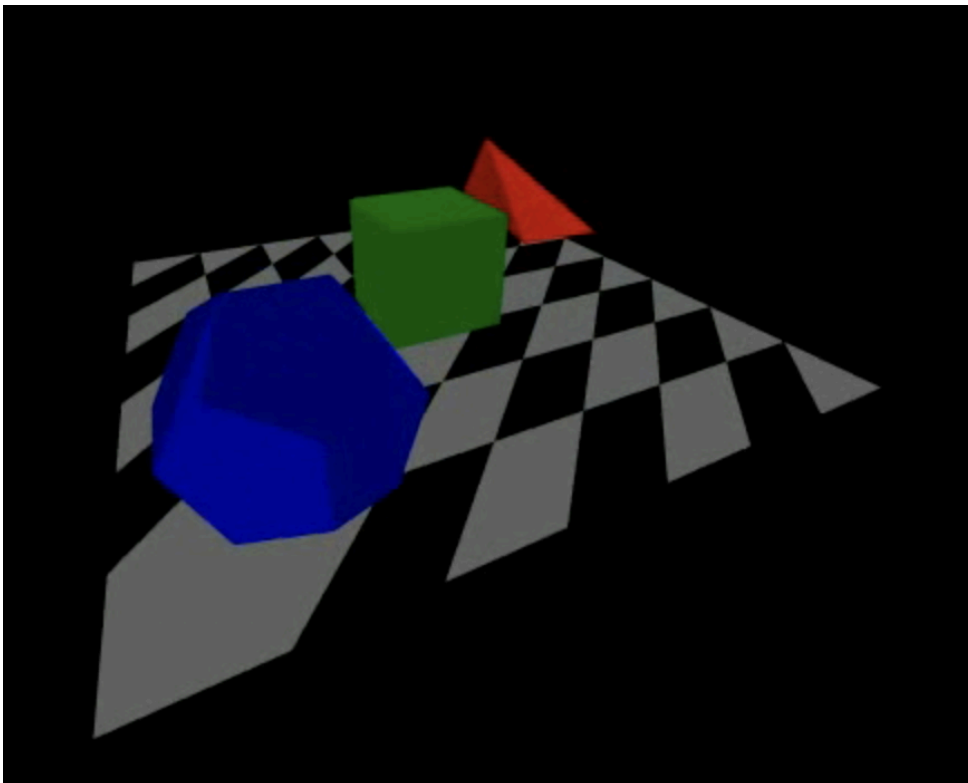
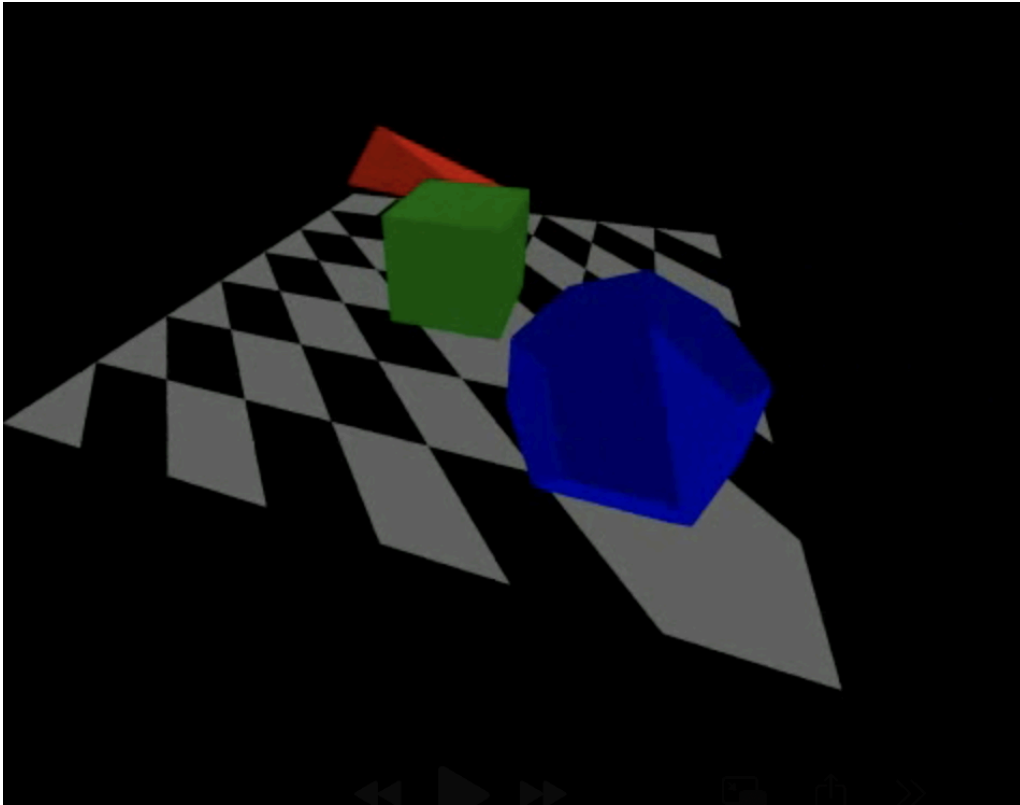
1. Есть прозрачность и отражение



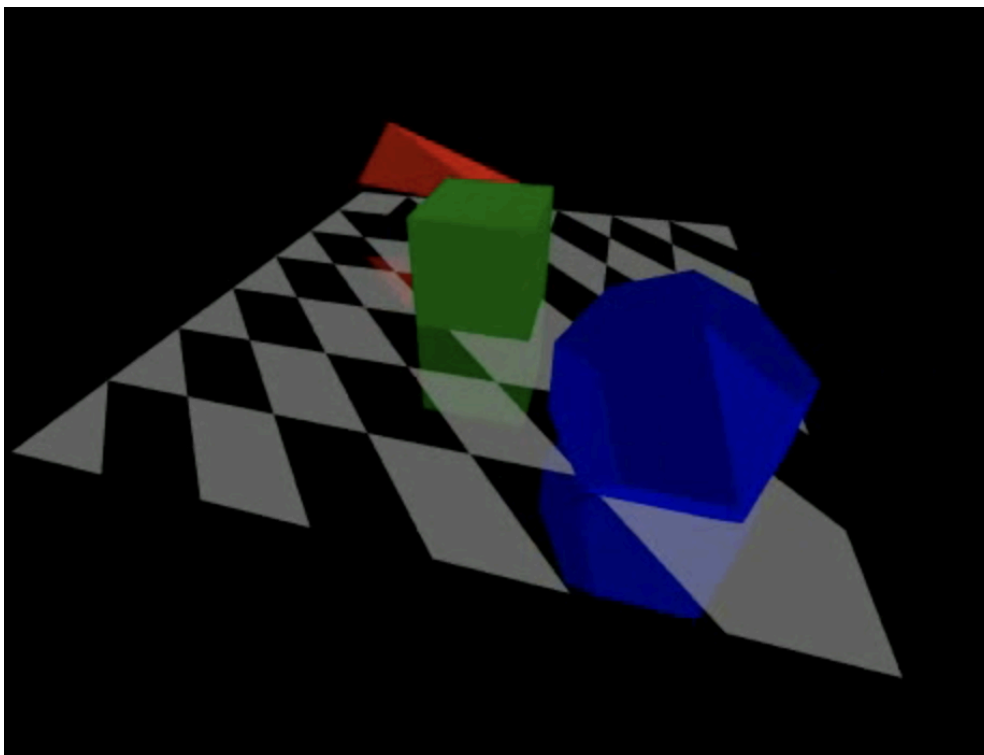
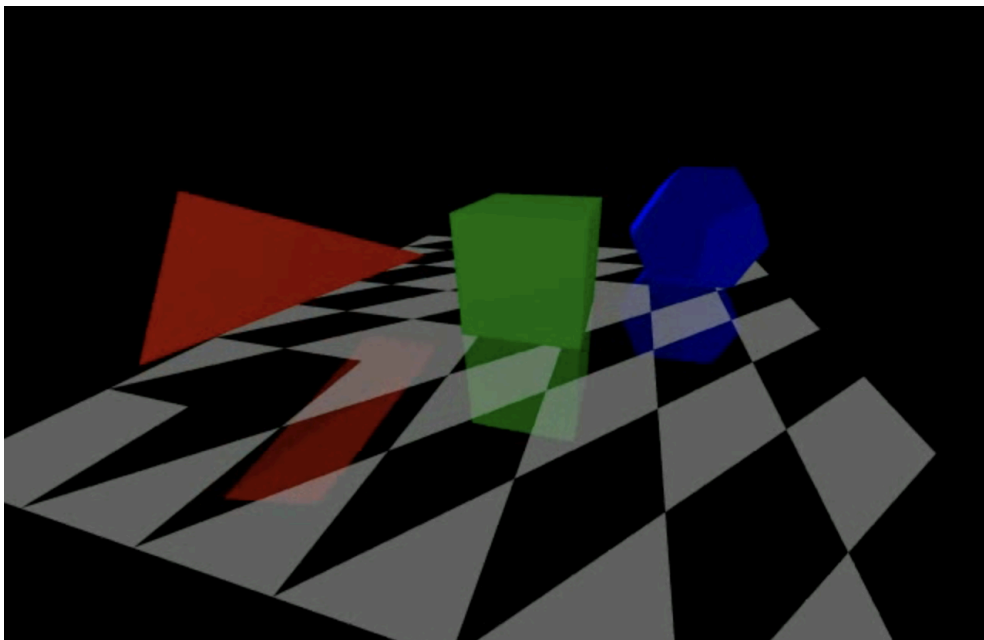


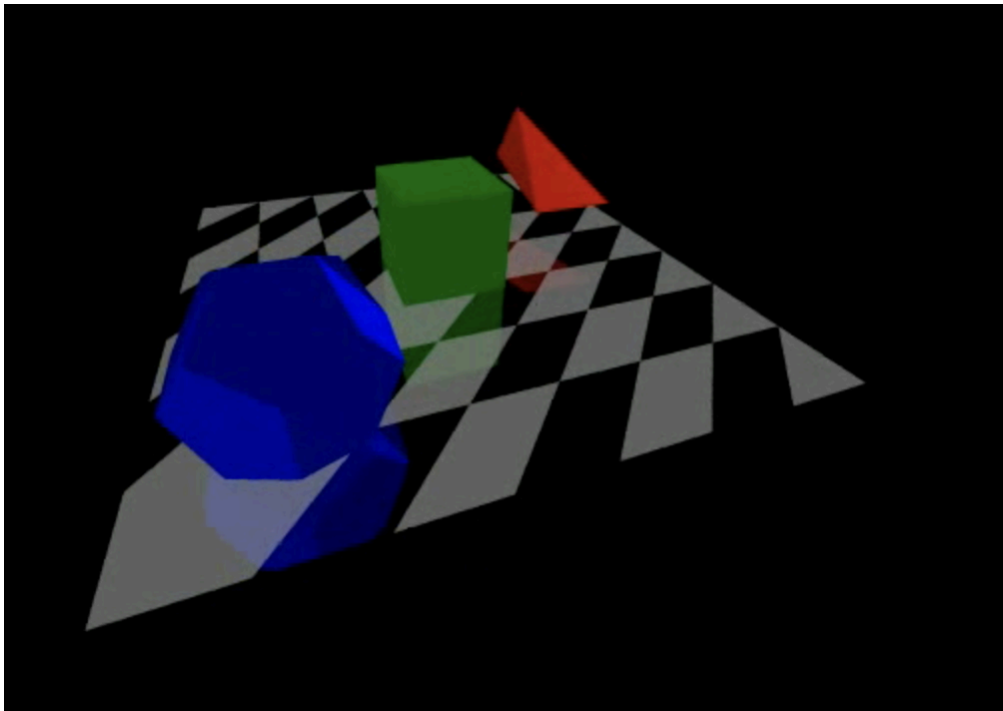
2. Нет ничего



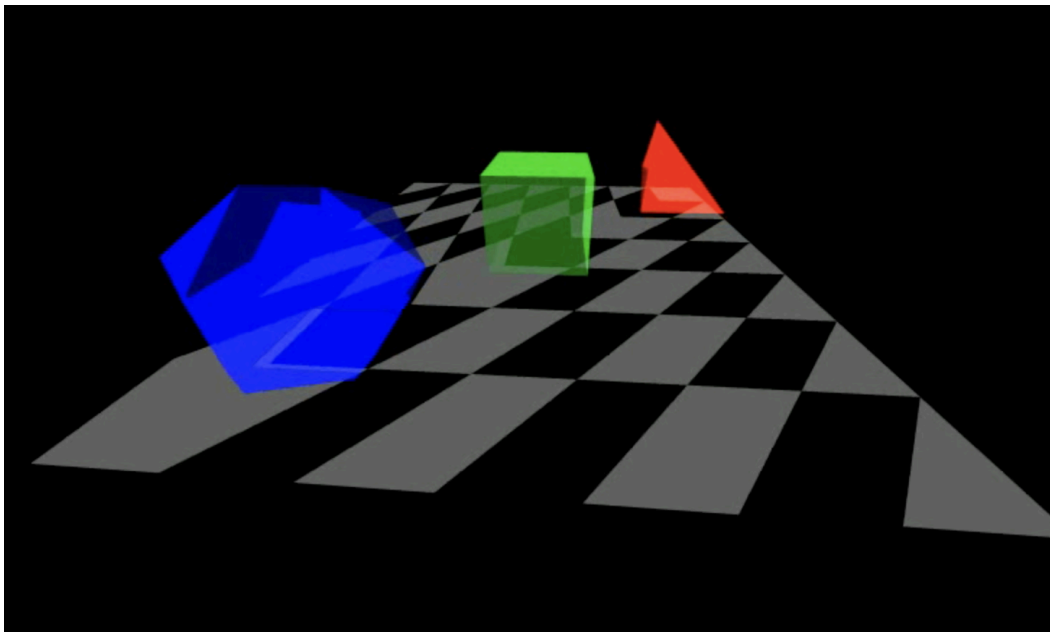


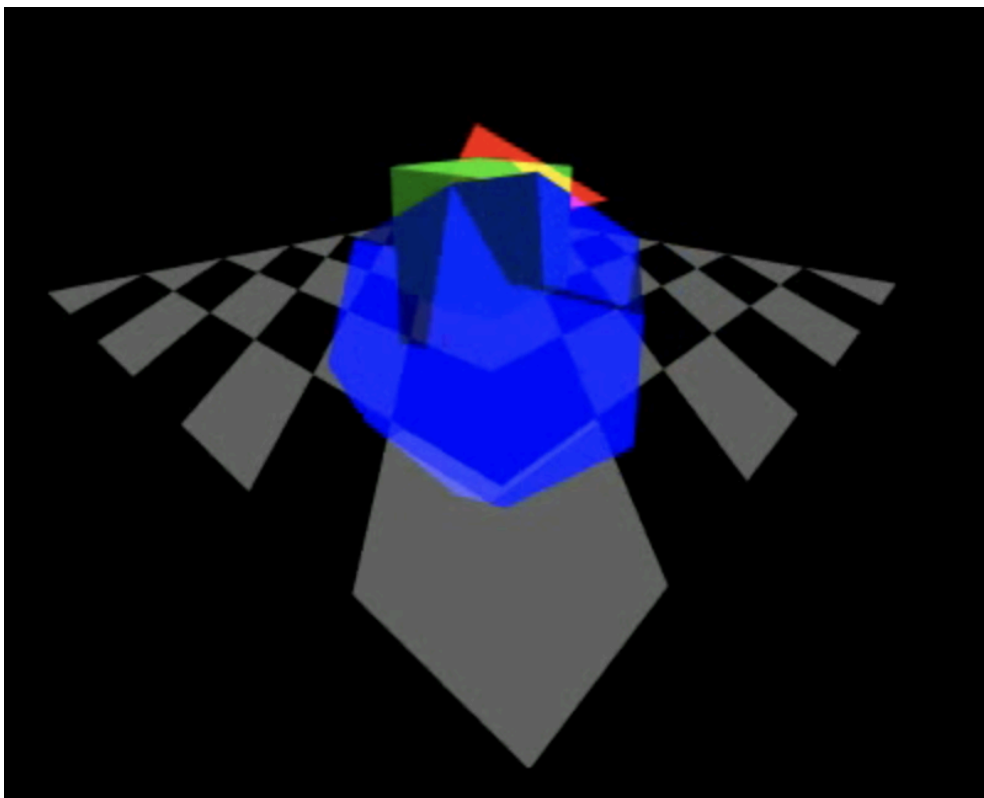
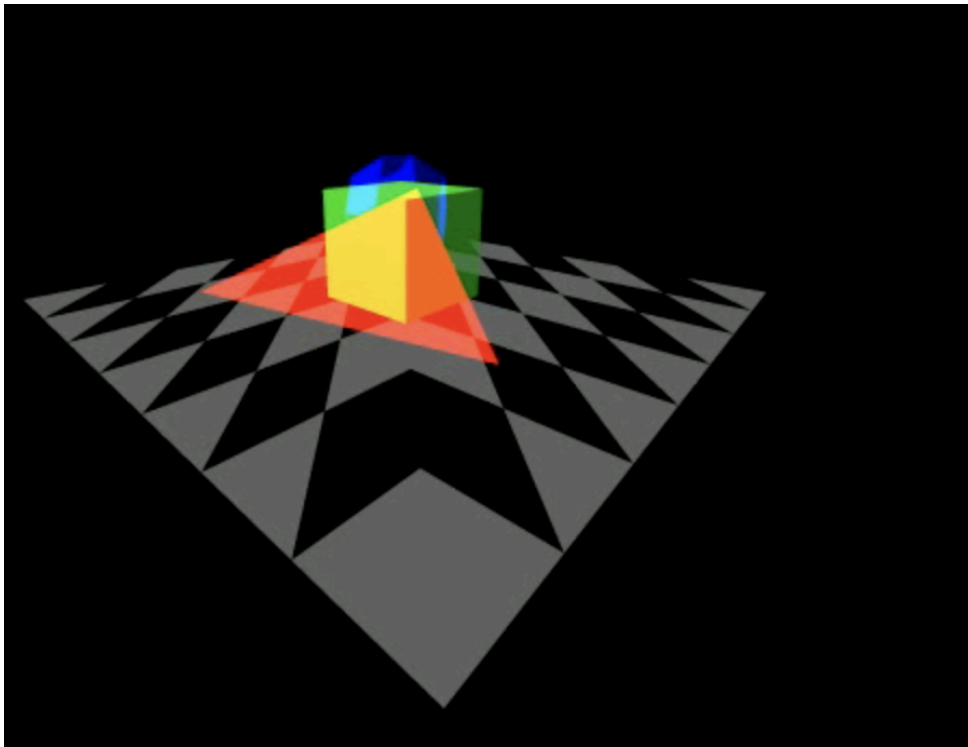
3. Есть отражение





4. Есть прозрачность





Пример default входных данных:

```
100
ress1/%d.data
800 800 120
7.0 3.0 0.0  2.0 1.0  2.0 6.0 1.0  0.0 0.0
2.0 0.0 0.0  0.5 0.1  1.0 4.0 1.0  0.0 0.0
3.0 3.0 0.5  1 0. 0.0  1.0
0.0 0.0 0.0  0. 1 0.  1.75
-3.0 -3.0 0.0  0.0 0. 1  1.5
-5.0 -5.0 -1.0  -5.0 5.0 -1.0  5.0 5.0 -1.0  5.0 -5.0 -1.0  1.0 0.9 0.25
3
8.0 0.0 3.0  1 0.1 1 0.6
1.0 5.0 3.0  0.1 0.1 0.1 0.7
1.0 5.0 4.0  0.1 0.1 0.1 0.3
0. 0.2 0. 0.
0.5 0.5 0.5 0.5
4
2
```

Выводы

В ходе выполнения курсовой работы была разработана программа для рендеринга трехмерных сцен методом трассировки лучей с поддержкой различных геометрических фигур, источников света, текстурирования и эффектов прозрачности и отражения. Реализованы версии алгоритма как для центрального процессора (CPU), так и для графического процессора (GPU), что позволило провести сравнительный анализ производительности.

В результате исследования было установлено, что использование GPU значительно ускоряет процесс рендеринга, особенно при увеличении разрешения изображения и сложности сцены. Это связано с параллельной обработкой множества лучей, которую эффективно выполняют графические процессоры.

Кроме того, благодаря применению текстурирования и различных визуальных эффектов, таких как отражение и прозрачность, удалось добиться фотореалистичного

отображения объектов сцены. Также была реализована система сглаживания (SSAA), что улучшило качество итогового изображения, устранив визуальные артефакты на границах объектов.

Таким образом, разработанная программа может быть использована в качестве основы для дальнейшего расширения функционала, включая поддержку более сложных материалов, динамических сцен и дополнительных визуальных эффектов.

Литература

1. <http://www.ray-tracing.ru/articles213.html> [дата обращения: 07.01.2025]
2. <https://habr.com/ru/articles/342510/> [дата обращения: 07.01.2025]